



Before you Start:

This is not a checklist, this guide has a lot of questions. This is an exhaustive, upper-bound sheet created for first and second year students, whose aim is to crack a good internship soon. DSA is a skill, and this skill is only achieved by doing this skill badly, DSA requires grind.

Never fall in the trap of “Solve these 10 DSA patterns and crack any FAANG”. This is the biggest clickbait in the industry and a lot of people fall for it.

Striver sheet is not enough for any OA (aadhi dhuniya striver karke baithi hai), if you are a first year then your priority should be to finish striver, you can use this sheet side by side for practice topics. If you are in 2nd year and your striver is still remaining then, your priority should be to finish striver asap (else you will be cooked), then you can practice from this sheet.

Hamesha video lectures pe depend rahena band kardo, CF, LC, CSES ke editorials padhke seekhne ki habit bana lo, this is very important.
Give a lot of contests, and always upsolve by editorials.

Btw Errichto is the GOAT!!

This sheet is for our juniors and their juniors, this sheet has been created and maintained by , for any edits contact to :

Krishna Mittal:	8770588119
Sachit Jain:	9910808660
Rudransh Pratap Singh:	9897353779

Pre - Final Year SMP Mentors, Batch of '28

In guidance of these OG people :

Ashish Jha [Linkedin](#)

Debopriyo Sen [Linkedin](#)

Harsh Sharma [Linkedin](#)

Batch of '27

DSA Prep

DSA is hands-down the most important aspect of Internship season. With the rise of AI generated slop and Vibe-coded projects, DSA becomes the only way companies can actually judge you.

You cannot clear a single OA without DSA.

MOST IMPORTANT: STOP USING CHAT-GPT OR ANY OTHER AI TOOL FOR DSA PREPARATION. USING AI FOR THE PURPOSE OF DE-BUGGING IS STRICTLY BARRED. THE GHOST OF YOUR AI SLOP WILL HAUNT YOU IN THE ACTUAL OA WHEN YOU DO NOT HAVE AI ASSISTANTS AND HAVE TO DEBUG YOUR OWN LOGIC.

Apart from striver sheet another underrated sheet is the CSES sheet. Hands down one of the best collections of problems available . it literally covers a lot of ideas which can form the building blocks of many problems so please refer to it once.

Our fav DSA sheets and youtube channels:

[Strivers A2Z DSA Course/Sheet - Crack Any FAANG or PBCs](#)

[Practice Problems | Code With Aryan | codeWithAryan](#)

[codestorywithMIK - YouTube](#)

[CodeNCode - Youtube](#)

One personal suggestion would be plz go and read cf blogs , they teach you so much and helps you brainstorm and look into multiple approaches of same problems ... so its highly recommended if we want to truly improve 😊

Array:

Arrays are the most fundamental topic and need a lot of practice. Arrays aren't about how many concepts you know; rather, it is about the number of problems and your ability to think out of the box, mathematically.

Start with Striver, **Easy and Medium Section**

Arrays are the most important topic as per OAs/Interviews are concerned.

Here are some of the subtopics that you need to practice for Array

1. Prefix Sum

<https://leetcode.com/problems/subarray-sum-equals-k/description/>
<https://leetcode.com/problems/contiguous-array/description/>
<https://leetcode.com/problems/binary-subarrays-with-sum/description/>
<https://leetcode.com/problems/make-sum-divisible-by-p/description/>
<https://leetcode.com/problems/number-of-submatrices-that-sum-to-target/description/>

2. Difference Array

<https://leetcode.com/problems/corporate-flight-bookings/description/>
<https://leetcode.com/problems/minimum-number-of-k-consecutive-bit-flips/>
<https://leetcode.com/problems/maximum-sum-obtained-of-any-permutation/>

3. In-Place array manipulation

<https://leetcode.com/problems/first-missing-positive/>
<https://leetcode.com/problems/wiggle-sort-ii/>

4. HashMap

<https://leetcode.com/problems/insert-delete-getrandom-o1/>
<https://leetcode.com/problems/4sum-ii/>
<https://leetcode.com/problems/subarray-sums-divisible-by-k/description/>

5. Kadane's Algo (most important)

<https://leetcode.com/problems/maximum-subarray-sum-with-one-deletion/description/>
<https://leetcode.com/problems/maximum-absolute-sum-of-any-subarray/>
<https://leetcode.com/problems/k-concatenation-maximum-sum/>

6. Coordinate Compression

<https://leetcode.com/problems/number-of-submatrices-that-sum-to-target/description/>
<https://codeforces.com/problemset/problem/816/B>
<https://codeforces.com/problemset/problem/1000/C>
<https://codeforces.com/problemset/problem/489/C>

7. Cyclic Sort

<https://leetcode.com/problems/first-missing-positive/>

8. Line Sweep

<https://cses.fi/problemset/task/1629>
<https://cses.fi/problemset/task/1619>
<https://leetcode.com/problems/my-calendar-iii/>

Arrays do not end here, you can only get better at arrays by solving loads of problems, there are a lot of patterns that do not even have a name... you need to keep solving problems.

Binary Search :

- First learn about this from striver sheet and solve some basic questions to learn about monotonicity and how to write check functions
- Then you can solve this sheet to get a good grasp of different varieties of questions : [Binary search questions](#)
- Binary search is often used in problems from other topics also so make sure you understand the basic principles well and have good grasp in writing the check function.
- After this you can move to solving USACO guide for more challenging problems : [usaco guide binary search](#)

For more problem you can refer these questions after completing this section from Striver Sheet : [Binary_Search_Problem_Set.docx](#)

Strings:

Most string problems are just array problems in disguise (i mean string is just an array of chars, right??)

So the string-specific part mostly is just **String Matching Algorithms** and their super wide usage

1. Complete Striver String Easy and Medium
2. More Questions => [String Questions](#)
3. Learn String Matching Algos
 - a. [String Hashing - Algorithms for Competitive Programming](#) (Goated Technique, has a lot of applications other than strings too)
 - b. [Rabin-Karp for String Matching - Algorithms for Competitive Programming](#)
 - c. [Prefix function - Knuth-Morris-Pratt - Algorithms for Competitive Programming](#)
 - d. [Manacher's Algorithm - Finding all sub-palindromes in O\(N\)](#) (Has a lot of applications in arrays too)
 - e. Practice from Striver Sheet, String (hard)

STL:

1. Do as much as in striver video
2. The rest you will learn by solving problems and you will develop your own way of utilising STL functions
3. And yeah if you see some new kinda syntax somewhere, just go and find out from GPT or smthng, what's that
4. [Parsing input using stringstream in C++ - Codeforces](#) (Useful in annoying string parsing questions)

Number Theory / Maths :

<https://www.youtube.com/playlist?list=PLtfqa971vD5FT6gYRxs0BDH-0WRz1FdCL> - hands down one of the best playlist available rn 😊

- Number theory can also serve as a small part in some questions involving gcd, lcm, SPF and factorization etc. so make sure you do it properly without mugging things up.
- Refer to the above lectures to learn all concepts that can be used in problems, it also covers problems from CF so you are good to go with xd.
- These are some very basic utility things you should keep in your pocket : [A Beginner's guide to Number Theory problems in Competitive Programming | by Khushi Keswani | Medium](#)
- If you prefer articles then this is a very good to go resource : [Number Theory for DSA & Competitive Programming - GeeksforGeeks](#)
- Sieve of eratosthenes resource : [Sieve of Eratosthenes](#)
- After this learn about SPF : [Sieve of Eratosthenes \[smallest prime factor spf\]](#)
- Refer to this lecture : [L8. Smallest Prime Factor \(SPF\) | Prime Factorisation | Query...](#)

For more problems refer to this doc : [Number_Theory_Problem_Set.docx](#)

Two Pointers & Sliding Window:

Two Pointers and Sliding Window are extremely important techniques for OAs and interviews. They are not difficult because of implementation — the difficult part is **recognizing when the problem can be reduced to maintaining two boundaries and exploiting some invariant between them.**

First, complete **ALL the relevant problems from Striver's Sheet** for Two Pointers and Sliding Window.

Do NOT jump directly to the problems below. Striver should be your foundation — the problems below are meant to expose you to harder variations and less-obvious applications of the same ideas.

Problems: [TwoPointer Problems](#)

Stacks, Queues:

- In stack the most important part is understanding monotonic stack and queues
- First complete this section from striver and get a good idea of the topic
- No need to implement this from scratch you can always use stl btw but it's good to know the internal implementation :)

- To learn about monotonic ds follow this article : [monotonic ds](#)
- For more challenging questions refer to USACO guide : [usaco stacks](#)

For more questions refer to this doc : [Stacks_Queues_Problem_Set](#)

Linked Lists :

Just Follow striver , it's enough for interviews and this topic is rarely asked in OAs.

Priority Queue:

Priority Queue is mainly about problems where you repeatedly need the **minimum / maximum element**, but the elements keep changing.

First, complete **ALL the relevant Priority Queue / Heap problems from Striver's Sheet**.

Do not skip the basic heap problems. You should know:

- Min Heap / Max Heap
- `priority_queue` in C++
- Push / Pop / Top
- Custom comparator
- Building a heap
- Heapify
- Heap Sort
- Time complexity of heap operations

Problems: [PriorityQueue](#)

Graphs: (list not necessarily to be followed in order)

1. Puri Striver ki playlist solve kr lena khud se, and uske videos bhi dekh lena, Graph thik se bataya hai usne
2. Tarjan Better than striver => [Bridges and Articulation Points | Bridge finding algorithm in Graphs](#)
3. Some Questions [LeetCode Graph Study Guide & Problem List](#)
4. 0/1 DFS => [0-1 BFS - Algorithms for Competitive Programming](#)
5. Some Questions (Slowed + Reverbed) [Graphs Roadmap](#)
6. CF Mashup (from Colin Galen) => <https://codeforces.com/contests/317667>
7. [More questions](#) 😊
8. More DSU, cuz why not -> <https://codeforces.com/edu/course/2/lesson/7/1>
9. Also solve **CSES** sheet for covering more Graph Patterns
10. Btw, one of my fav DSU Problems => [Bricks Falling When Hit - LeetCode](#)
11. For Graphs oriented to CP, look into Colin Galen Topic Streams and Errichto Streams
12. Also feel free to look into USACO or CP algo for explanations of specific algos

13. Some more questions of Graphs paired with DP are also in the DP section

Dynamic Programming:

Dynamic Programming is one of the most important topics for competitive programming and interviews. **Do NOT start these problems directly.**

DP FROM STRIVER IS NOT RECOMMENDED... PEHELE MEMO, TABULATION THIS METHOD DOES NOT WORK... U NEED TO START VISUALIZING HOW DP TABLE IS FILLING. WE RECOMMEND YOU DO DP THEORY FROM:

https://www.youtube.com/playlist?list=PL_z_8CaSLPWekqhdCPmFohncHwz8TY2Go

Then: https://www.youtube.com/playlist?list=PLqf9emQRQrnKA_EeveiXQj_uP25w8_5qL

First, complete **ALL the Dynamic Programming problems from Striver's DP Sheet**, including 1D DP, 2D/3D DP, DP on Subsequences, DP on Strings, DP on Stocks, DP on LIS, MCM/Interval DP and DP on Squares. Striver's sheet already covers the fundamental DP patterns that you are expected to know.

Once you have completed Striver, use the following problems to go beyond the standard patterns and learn how DP appears in harder problems.

Do not memorize DP solutions. For every problem, focus on:

- What is the state?
- What does the state represent?
- What are the transitions?
- What are the base cases?
- Can the state be reduced?
- What is the time and space complexity?

In DP the most important thing is defining your state and transition once you do this this can solve your question or can completely take you in wrong direction...so think it wisely 😞

For the Problems refer this doc: [DP Problems](#)

(PS: we do not recommend learning theory from striver, but do solve the problems in the striver sheet. Hence this doc references the striver problems... the problem distribution in striver is good)

ALSO U CAN USE THIS:

<https://www.interviewbit.com/courses/programming/dynamic-programming/>

DP does not end here. **DP is not a collection of 15–20 templates.** Start solving **random 1500–2000 rated DP problems on Codeforces/CSES/USACO/LeetCode.**

Segment Trees :

- For learning segment trees follow this playlist : [segment tree](#)
- Now you know all the concepts that can be used in segment tree problems
- Now start solving CSES section of this topic
- For more problems refer to this sheet : [Segment_Tree_Problem_Set.docx](#)
- Segment trees appear usually in the 4th question of LC contests and it serves as a helper in some other questions most of the time so make sure you do it properly.
- If you want to read some article then this is a very good article to read : [Segment Tree - Algorithms for Competitive Programming](#)
- Segment tree + coordinate compression is another important concept which is covered in the playlist so make sure to study it well , this idea is used in many problems. I was asked this in my internship interview as well lol!!

Tree:

1. Study Trees from Striver (Covers basics well) (Sheet / Videos)
2. Practice => Some (tbh a lot) Problems: [Binary Tree & BST Prblms](#)
3. Tree DP (very interesting):
 - a. You might want to refer to this: [DP on Trees Tutorial - Codeforces](#)
 - b. <https://usaco.guide/gold/dp-trees?lang=cpp>
 - c. Gem 💎 Alert => https://youtube.com/playlist?list=PLb3g_Z8nEv1j_BC-fmZWVHFe6jmU_zv-8s&si=vfkuxsp3vvhb05Awf
 - d. Good CF Problems => [DP on Trees & DAGs | Dynamic Programming on Trees & Graphs | Master Graph DP for DSA & Competitive Programming - YouTube](#)
 - e. [More questions](#)
 - f. RerootingDP => [Introduction to Re-rooting DP for Competitive Programming. \(like never befo...](#)
 - g. BinaryLifting 📦 => [Introduction | Binary Lifting | DP On Trees | Concepts & Questions - 1 | code...](#)
4. Also Don't you forget Tree CSES (you may skip the questions will super less submission rate though)

Now it's time to challenge yourself and learn new techniques 😞

In some questions, you will have to learn new techniques but don't stop brainstorming bcz that is the real fun :)

- [Pikachu](#)
- [Deoxys](#)
- [Rayquaza](#)
- [Groudon](#)

- [Latios](#)
- [Alakazam](#)
- [Ditto](#)
- [Zoroark](#)
- [Machop](#)
- [Zacian](#)
- [Mewtwo](#)
- [Gengar](#)
- [Charizard](#)
- [Mega Rayquaza](#)
- [Snorlax](#)
- [Alakazam](#)
- [Slowking](#)
- [Tropius](#)
- [Porygon-Z](#)
- [Rotom](#)
- [Garchomp](#)
- [Zygarde](#)
- [Dragonite](#)
- [Metagross](#)
- [Yveltal](#)

Zinda Ho ??

